



**Team
AXIOMATA**

OFFLINE SYNC AND FIELD OPERATIONS

Developed By
Team AXIOMATA

Developed for
Waypoint Group (Pvt) Ltd.

Access to RELAY

1. The Challenge

Waypoint's hill-country delivery routes - particularly VEH057's Kandy-to-Nuwara-Eliya run - lose mobile signal for extended stretches. In the demo, Sunil's van enters a dark corridor where the driver must continue recording arrivals, deliveries, photos and problem reports without connectivity. Meanwhile the dispatcher cannot leave the screen blank or frozen: she needs to know the van is unreachable, see its estimated position and queue any necessary order changes. When signal returns, every record must land exactly once, at the time it actually happened, and any conflicting decisions must be reconciled by a clear rule. The dock shares this need on a smaller scale when a bay's Wi-Fi drops.

2. Core Principles

- The server is the source of truth. The phone holds the last server view plus its own unsent records, overlaid on screen.
- Every field write is an event with a `client_event_id` created before anything is sent.
- Records keep the time they happened. The server stores the device's virtual time and moves the workspace clock forward to meet it - never back.
- Physical facts win. A delivery recorded offline beats a plan change queued while the van was dark.
- Honest states. Every face shows what it knows and no more: the phone says 'No signal · N on phone'; the dispatcher sees an estimated stop and 'No signal since HH:MM'.

3. The Outbox (Browser Side)

Every driver and dock write goes through a local outbox stored in IndexedDB (Dexie, database 'relay', table 'outbox'). Each row holds:

Field	Meaning
<code>id (client_event_id)</code>	Random ID generated before the write
<code>type, payload</code>	Event type (e.g. <code>stop.complete</code>) and its data
<code>at</code>	Device virtual time when it happened (naive ISO string)
<code>offline</code>	True if recorded while <code>isOnline()</code> was false
<code>status</code>	<code>pending</code> → <code>synced</code> (applied/duplicate) or <code>rejected</code>
<code>createdAt</code>	Real time in ms - determines send order
<code>attempts</code>	Failed sends so far; a failed request never drops the row

3.1 Recording

`record(type, payload, label, atMs)` picks the timestamp via `snapTo(atMs)`: the suggested time if it is later than the device clock (advancing it), otherwise the clock's current time. The row is written to IndexedDB as 'pending' and `flush()` is started without waiting - the screen never waits for the network.

3.2 Flushing

`flush()` runs at most one copy per tab. While online it reads pending rows in `createdAt` order (up to 40 per batch), posts them to `POST /api/sync` with the device ID and `pending_after` count, then writes results and advances the local clock. Triggers: after every `record()`, on the 'online' browser event, on `visibilitychange`, every 10 seconds, and from the 'Sync now' button.

3.3 Heartbeat

While online, the driver app posts `{pending: N}` to `POST /api/driver/heartbeat` every 15 seconds. The server records the last-seen time and uses it to detect dark vans.

4. Dispatcher View of a Dark Van

The server detects a dark van when a human-driven, running trip's phone has been silent for `HEARTBEAT_DARK_S` (default 45 s). The trip's signal flag is set to 'dark' and a notice is pushed to the dispatcher. The canvas shows the van in amber with 'No signal since HH:MM' and an estimated stop based on planning-standard travel from the last confirmed position.

If the dispatcher tries to move an order belonging to a dark van, `move_order()` queues a `pending_move` instead of applying it immediately. The van's driver is told when connectivity returns.

5. Server-Side Sync (`POST /api/sync`)

`sync()` in `routers/field.py` applies events in order. For each event:

- 1. Check for an existing event row with the same `(workspace_id, client_event_id)`. If found → duplicate; return the stored result.
- 2. Run the domain function inside a savepoint; record `sync.<type>` with `client_event_id`, `device_at` and the result.
- 3. If the domain function rejects → record `sync.rejected.<type>` with `result={rejected: message}`. A retry with the same id returns this answer and is never applied.
- 4. The unique constraint `uq_event_client_id` catches two concurrent copies of the same record.

6. Conflict Resolution — Physical Facts Win

When a dark van comes back online, `heartbeat()` calls `apply_pending_moves()`. It waits until the device reports no unsynced records (`pending == 0`), ensuring offline deliveries are applied first. Each queued move is then either applied or cancelled:

- Applied: the stop is still undelivered and the target passes `check_target()`.
- Cancelled (with a resolution text): the stop was already delivered offline, or the move no longer fits. The dispatcher sees 'Conflict resolved: delivery kept'.

`complete()` also cancels any queued `pending_move` for orders it delivers. The rule is explicit and tested: physical field facts always override dispatch changes.

7. 'Test Offline Mode' for Demonstrations

The driver app's Me tab includes a 'Test offline mode' toggle (setSimulatedOffline()). While on, isOnline() returns false even though the radio is active - the outbox stops flushing, heartbeats stop, and the live SSE stream closes. To the server this is indistinguishable from a real dead zone, so the van goes dark for real and the dark corridor scenario can be demonstrated safely without airplane mode.

8. What Each Face Sees

Face	Offline State Shown
Driver	'No signal · N on phone' pill; 'You're offline. Keep working.' banner; Sync tab with pending count and per-record status
Dispatcher	Amber van marker; 'No signal since HH:MM'; estimated stop; queued move notice
Loader (dock)	'Offline · N saved' pill; 'No connection at the dock. Keep loading.' bar; flags show 'Saved on this tablet'
Store manager	'Estimated' arrival times while the van is dark